RAJAGIRI COLLEGE OF SOCIAL SCIENCES

(AUTONOMUS)

KALAMASSERY

Department of Computer Science

Full Stack Web Development using MERN

OPERO - PROJECT MANAGEMENT APP - Project Report

Done by

EDWIN K JOHNSON

Reg No: 24117016

2024-2028

RAJAGIRI COLLEGE OF SOCIAL SCIENCES

(AUTONOMUS)

KALAMASSERY

CERTIFICATE

Opero - Project Management App

Certified that this is the bonafide record of project work done by

EDWIN K JOHNSON

Reg No:24117016

Bachelor of Computer Application
of
Rajagiri College of Social Sciences(Autonomus)

Project Coordinator
name

Head of the Department
Prof. Bindiya M Varghese

Internal Exminer

External Examiner

ACKNOWLEDGEMENT

With heartfelt gratitude, I extend my deepest thanks to the Almighty for His unwavering grace and blessings that have made this journey possible. May His guidance continue to illuminate my path in the years ahead.

I am immensely thankful to Prof.Bindiya M Varghese, head of the department of Computer Science , my Faculty Guide and Prof.Ann Rija Paul my dedicated Project Coordinator, for their invaluable guidance and timely advice, which played a pivotal role in shaping this project. Their guidance, constant supervision, and provision of essential information were instrumental in the successful completion of the Project.

I extend my profound thanks to all the professors in the department and the entire staff at RCSS for their unwavering support and inspiration throughout my academic journey. My sincere appreciation goes to my beloved parents, whose guidance has been a beacon in every step of my path.

I am also grateful to my friends and individuals who generously shared their expertise and assistance, contributing significantly to the fulfillment of this endeavor.

EDWIN K JOHNSON

ABSTRACT

The Team Collaboration and Project Management System is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) with real-time communication capabilities powered by Socket.io. The primary objective of this system is to provide an efficient platform for teams to manage projects, assign tasks, and communicate seamlessly within a centralized environment.

The application allows users to register and log in securely using JWT-based authentication. Role-based access control is implemented to differentiate between administrators and regular users. Once authenticated, users can create projects, add team members, assign tasks, update task statuses, and monitor project progress. The system follows a RESTful API architecture and uses MongoDB for flexible and scalable data storage.

A key feature of the application is its real-time chat functionality. Using WebSocket technology through Socket.io, team members can communicate instantly within project-specific chat rooms. Messages are stored in the database to ensure persistence and future reference.

The system is designed with a modular backend architecture, ensuring scalability, maintainability, and security. Environment variables are managed using dotenv to protect sensitive information such as database credentials and secret keys.

Overall, this project demonstrates the implementation of modern web development practices, real-time communication, secure authentication, and structured API design to build a production-ready collaborative platform suitable for academic and practical applications.

LIST OF TABLES

2.1 Software Requirements for Opero Project Management System 12

4.1 Unit Test Cases for Opero Project Management System 30
4.2 Integration Test Cases for Opero Project Management System 31
4.3 System Test Cases for Opero Project Management System 32
4.4 Performance Test Cases for Opero Project Management System 33

LIST OF FIGURES

2.1 Project Pipeline 9

3.1 Use case Diagram 17
3.2 activity diagram 18
3.3 sequence diagram 20
3.4 class diagram 21
3.5 Login Page 24
3.6 Registration Page 24
3.7 Admin Dashboard 25
3.8 Manager Dashboard 25
3.9 Project Creation Page 26
3.10 Kanban Task Board 26
3.11 Task Management Page 27
3.12 Project Chat Interface 27
3.13 databases in MongoDB 28

5.1 src files 34

6.1 Builds Uploaded 35
6.2 Files and folders inside the builds 35

CONTENTS

ACKNOWLEDGEMENT **i**

ABSTRACT **ii**

LIST OF TABLES **iii**

LIST OF FIGURES **iv**

1 INTRODUCTION **1**

2 SYSTEM ANALYSIS **3**

2.1 Module Description 3
2.2 Business Rules 5
2.3 Analysis of Architecture (and/ or) Algorithms 7
2.4 Project Pipeline 9
2.4.1 Software Requirement Specification (SRS) 10
2.5 Feasibility Analysis 12
2.5.1 Technical Feasibility 12
2.5.2 Economical Feasibility 13
2.5.3 Operational Feasibility 13
2.6 System Environment 14
2.6.1 Software Environment 14
2.6.2 Hardware Environment 15
2.7 Actors and Roles 16

3 SYSTEM DESIGN **17**

3.1 Use case model 17
3.2 Activity Diagram 18
3.3 Sequence Diagram 20
3.4 Class Diagram 21
3.5 Project Design 23
3.5.1 UI Design 23
3.5.2 Database Design 28

4 TESTING **29**

4.1 Unit Testing 29
4.2 Integration Testing 31
4.3 System Testing 32

4.4 Performance Testing 33

5 DEPLOYMENT **34**

5.1 Folder Structure 34

6 GIT HISTORY **35**

7 CONCLUSIONS **36**

1 INTRODUCTION

Project management plays a vital role in modern organizations, particularly in software development and information technology industries where collaboration, time management, and efficient resource allocation are essential for success. As projects increase in complexity and teams become more distributed, managing tasks, monitoring progress, ensuring accountability, and enabling seamless communication have become increasingly challenging. Traditional project management methods such as spreadsheets, emails, and manual reporting systems are often inefficient and lack real-time collaboration capabilities.

With the rise of agile methodologies and collaborative work environments, organizations require modern digital solutions that provide transparency, adaptability, and continuous workflow monitoring. Agile practices emphasize iterative progress, task visibility, and team communication. To support these practices, web-based project management systems have become essential tools for improving productivity and coordination among teams.

Opero is a full-stack Web-Based Project Management and Team Collaboration System developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), integrated with real-time communication using Socket.io. The system is inspired by industry-standard tools such as Jira and Trello, but extends functionality by combining structured project management with integrated team communication in a single platform.

The frontend of Opero is developed using React.js, implementing a modern Single Page Application (SPA) architecture. The user interface is designed to be clean, responsive, and intuitive, providing dashboards tailored to different user roles. The application supports multiple roles, including Admin, Manager, and Team Member, with clearly defined access permissions and responsibilities. Role-Based Access Control (RBAC) ensures that users can only access features relevant to their role.

The backend is built using Node.js and Express.js, following a modular RESTful API architecture. It handles authentication, project management, task operations, user management, and message storage. Secure authentication is implemented using JSON Web Tokens (JWT), ensuring protected routes and secure session handling. Passwords are encrypted using bcrypt hashing techniques to enhance security.

MongoDB is used as the database system, providing flexible and scalable storage of users, projects, tasks, and chat messages. The database design follows schema-based modeling using Mongoose, enabling efficient data validation and structured relationships between entities.

A key feature of Opero is its real-time communication system powered by Socket.io. Each project includes a dedicated chat room where team members can communicate instantly. This feature enhances collaboration by allowing real-time discussions, quick updates, and immediate issue resolution without leaving the platform. Chat messages are stored in the database for persistence and future reference.

The system also incorporates a Kanban-style task management board, where tasks move

through workflow stages such as Pending, In Progress, and Completed. This visual representation improves transparency, helps identify bottlenecks, and enables efficient progress tracking.

Opero demonstrates the practical implementation of modern web development technologies, secure authentication mechanisms, RESTful API design, real-time WebSocket communication, and scalable NoSQL database integration. The project serves both as an academic learning exercise and as a production-ready collaborative system that reflects real-world software engineering practices.

2 SYSTEM ANALYSIS

2.1 Module Description

The Opero – Project Management and Team Collaboration System is divided into multiple functional modules, each playing a significant role in ensuring smooth project handling, secure user authentication, structured task tracking, and real-time team communication. The system is designed using a full-stack architecture, combining frontend interaction, backend processing, database management, and real-time messaging to provide an integrated collaborative environment. The main modules include:

- **Authentication Authorization Module:**This module is responsible for handling secure user registration and login processes. It verifies user credentials and generates JSON Web Tokens (JWT) for authenticated sessions. Passwords are encrypted using bcrypt hashing before being stored in the database, ensuring data security and protection against unauthorized access. The module also implements Role-Based Access Control (RBAC), which defines different permissions for Admin, Manager, and Team Member roles. Protected routes prevent unauthorized users from accessing restricted functionalities, ensuring that only authenticated users can interact with project and task data.

- **User Management Module:**This module manages all user-related operations within the system. It allows administrators to create, view, update, and manage user accounts efficiently. Each user is assigned a specific role that determines their level of access and responsibility within the platform. The module ensures that user data such as name, email, password, and role are securely stored in the MongoDB database. It also supports retrieving user information when assigning members to projects and tasks, thereby maintaining structured collaboration among team members.

- **Project Management Module:**The Project Management Module enables users to create, update, view, and delete projects within the system. Each project acts as a centralized workspace where tasks and team discussions are organized. The module ensures that only authorized users can modify project details or manage project members. It maintains structured project data, including project name, description, assigned members, and timestamps. By organizing tasks and communication under specific projects, this module ensures clarity and systematic workflow management.

- **Task Management Module:**This module is responsible for handling task creation, assignment, tracking, and status updates. Tasks can be assigned to specific team members and categorized based on workflow stages such as Pending, In Progress, and Completed. The system follows a Kanban-style task management approach, where tasks are visually represented and moved across different stages of completion. This visual workflow

improves transparency, helps identify delays or bottlenecks, and ensures that responsibilities are clearly defined. The module ensures that updates are reflected dynamically in the user interface for smooth interaction.

- **Real-Time Communication Module:**This module provides real-time chat functionality within each project workspace. It is implemented using Socket.io, which enables WebSocket-based two-way communication between the client and server. When users join a project, they automatically enter a dedicated chat room associated with that project. Messages sent by one team member are instantly broadcast to other members in the same project without requiring a page refresh. Additionally, chat messages are stored in the MongoDB database to ensure persistence and future reference. This module enhances collaboration by allowing instant communication directly within the project environment.

- **REST API Backend Processing Module:**This module handles all server-side operations and business logic. Built using Node.js and Express.js, it processes incoming requests from the frontend and performs appropriate operations such as data validation, authentication verification, and database interaction. The system follows a modular RESTful API architecture, where routes are separated for authentication, users, projects, tasks, and messages. Controllers manage the logic, while middleware ensures secure and structured request handling. This design enhances scalability, maintainability, and system organization.

- **Database Management Module:**The Database Management Module is responsible for storing and managing application data using MongoDB. Collections are maintained for users, projects, tasks, and chat messages. Mongoose schemas define data structure, validation rules, and relationships between entities. The module ensures that timestamps such as creation and update times are recorded automatically. MongoDB's flexible document-based structure allows efficient storage and retrieval of dynamic project and task data, supporting scalability and performance.

- **User Interface Dashboard Module:**This module manages the frontend interaction of the system, developed using React.js as a Single Page Application (SPA). It provides responsive dashboards tailored to different user roles. The interface includes project overview panels, task boards (Kanban view), user management screens, and chat windows. The module ensures smooth navigation, intuitive design, and real-time updates based on backend responses. By maintaining a clean and interactive UI, this module enhances user experience and engagement.

- **Environment Configuration Security Module:**This module ensures secure configuration management using environment variables. Sensitive information such as database connection strings and JWT secret keys are stored in environment files using dotenv, rather than being hardcoded in the application. This improves system security and allows easy deployment across different environments, such as development and production.

2.2 Business Rules

Business rules are specific guidelines, conditions, or constraints that govern the operations of a system. They define how processes and activities should be conducted to ensure consistency, security, and alignment with organizational objectives. Business rules ensure that the system behaves predictably and maintains data integrity while supporting collaboration and structured workflow management.

User Authentication and Role Management

- All users must register with a valid name, email address, and password before accessing the system.
- Passwords must be securely encrypted using bcrypt hashing before being stored in the database.
- Users must log in using valid credentials to receive a JSON Web Token (JWT) for authenticated access.
- The system implements Role-Based Access Control (RBAC), supporting three roles: **Admin, Manager, and Team Member**.
- Only authenticated users with valid tokens can access protected routes and perform authorized operations.
- Users can only access features and data permitted by their assigned role.

Project Management

- Only authorized users (Admin or Manager) can create, update, or delete projects.
- Each project must have a unique identifier and contain essential details such as project name, description, and assigned members.
- Projects can only be accessed by users who are assigned as members of that specific project.
- Project members can view project details and associated tasks but cannot modify project settings unless permitted by their role.
- When a project is deleted, all associated tasks and messages must be handled according to system integrity rules.

Task Management

- Tasks must be associated with a specific project and cannot exist independently.
- Each task must include a title, description, assigned member, and status.
- Task statuses follow a structured workflow: **Pending, In Progress, and Completed**.

- Only authorized users can create, update, assign, or delete tasks within a project.
- Task updates must reflect immediately in the system to ensure transparency and accurate progress tracking.

Real-Time Communication

- Each project contains a dedicated chat room accessible only to its assigned members.
- Messages sent within a project are broadcast in real-time using WebSocket communication.
- Chat messages must be stored in the database to ensure persistence and future retrieval.
- Users cannot send messages in projects where they are not registered members.
- Real-time updates must not require page refresh to ensure seamless collaboration.

Data Integrity and Security

- All sensitive data, including passwords and authentication tokens, must be securely managed and protected.
- Environment variables such as database connection strings and JWT secret keys must not be hardcoded in the application.
- The system must validate all incoming requests to prevent unauthorized data manipulation.
- Database operations must ensure consistency and prevent unauthorized access or modification.
- Version control fields (such as document version keys) must maintain data consistency during updates.

2.3 Analysis of Architecture (and/ or) Algorithms

MERN Stack Architecture with Real-Time Communication

Opero is developed using the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js. The system follows a modular full-stack architecture that ensures scalability, maintainability, and secure communication between the frontend and backend. Additionally, real-time communication is implemented using Socket.io to enhance collaboration among team members.

1. **Frontend Architecture – React.js:**
   - Opero uses React.js to implement a Single Page Application (SPA), allowing dynamic content updates without reloading the page.
   - The component-based architecture ensures reusable UI elements such as dashboards, task boards, chat interfaces, and project panels.
   - React Router enables smooth navigation between login, dashboard, projects, and task views.
   - State management ensures real-time UI updates when tasks are created, updated, or moved across workflow stages.
2. **Backend Architecture – Node.js and Express.js:**
   - Node.js provides the server-side runtime environment for handling asynchronous operations efficiently.
   - Express.js is used to design RESTful APIs for authentication, user management, project handling, task operations, and messaging.
   - The backend follows a modular structure separating routes, controllers, models, and middleware to maintain clean architecture.
   - Middleware functions handle request validation, authentication verification, and error management.
3. **Database Architecture – MongoDB with Mongoose:**
   - MongoDB is used as a NoSQL database to store users, projects, tasks, and chat messages in document format.
   - Mongoose schemas define structured data models with validation rules and relationships between collections.
   - Automatic timestamping ensures tracking of record creation and updates.

- The flexible document-based structure supports scalability and efficient handling of dynamic project data.

4. **Authentication and Security Mechanism:**
   - JSON Web Tokens (JWT) are used to implement secure authentication and protected routes.
   - Passwords are encrypted using bcrypt hashing before being stored in the database.
   - Role-Based Access Control (RBAC) ensures that Admin, Manager, and Team Member roles have appropriate system permissions.
   - Environment variables managed using dotenv protect sensitive data such as database credentials and secret keys.
5. **Real-Time Communication – Socket.io:**
   - Socket.io enables WebSocket-based real-time communication between the client and server.
   - Each project includes a dedicated chat room where team members can exchange messages instantly.
   - Messages are broadcast in real-time without page refresh, ensuring seamless collaboration.
   - Chat messages are stored in the database to maintain persistence and future reference.

MERN Stack for Opero?

The MERN stack provides a unified JavaScript-based development environment for both frontend and backend, ensuring efficient data flow and reduced development complexity. Its scalability, real-time capabilities, and modular architecture make it highly suitable for developing Opero as a secure, responsive, and collaborative project management system.

2.4 Project Pipeline

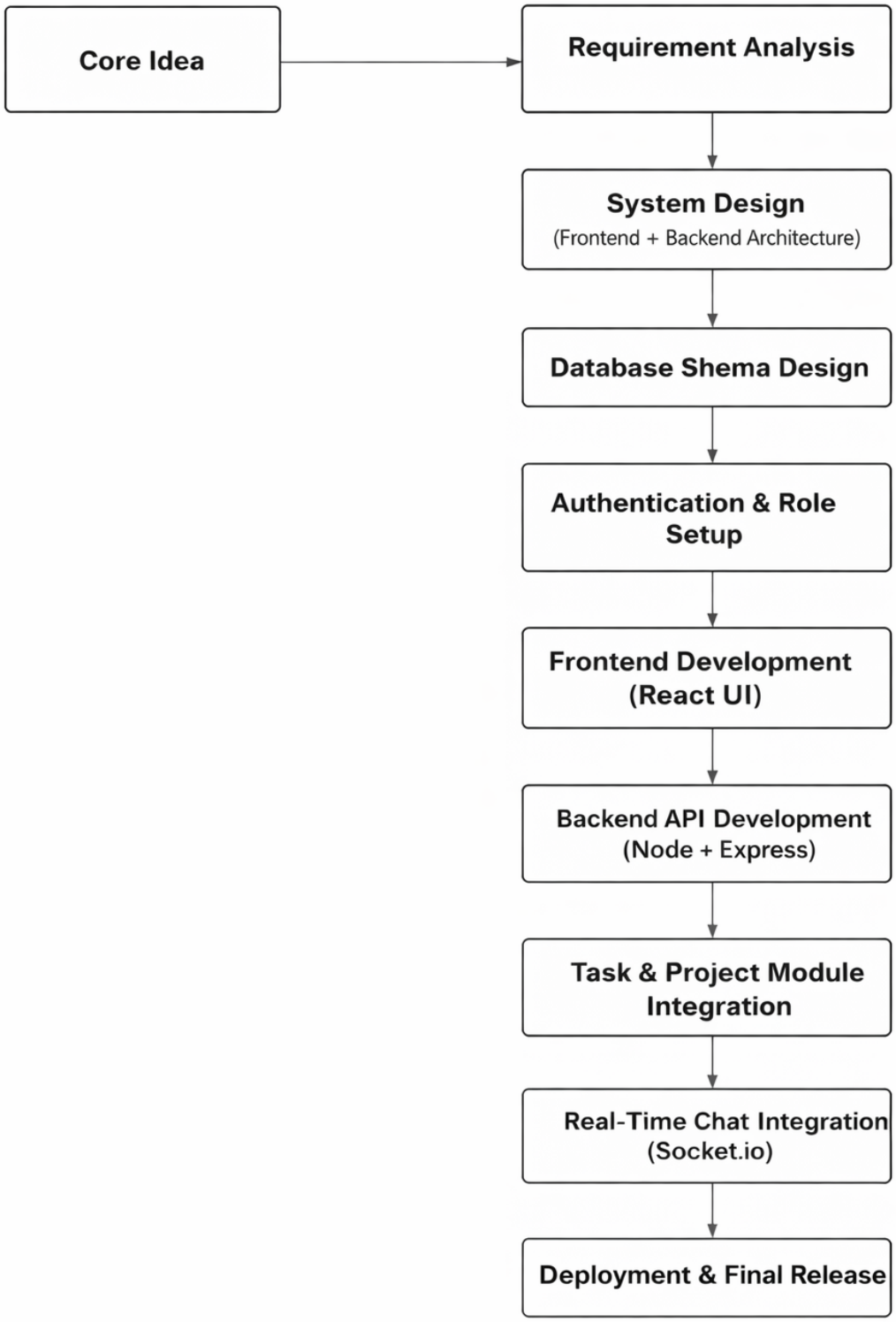


Figure 2.1: Project Pipeline

2.4.1 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the Opero Project Management System. It provides a detailed description of the system features, expected behavior, and constraints that guide the development process.

The purpose of the SRS document is to clearly specify what the system should accomplish and how it should operate from the user's perspective. It acts as a reference for developers, testers, and stakeholders to ensure that the final system meets the intended objectives.

The Opero system is designed to support collaborative project management by enabling users to create projects, assign tasks, track progress, and communicate effectively within teams. The system includes multiple user roles such as Admin, Manager, and Team Member, each with specific responsibilities and permissions.

Functional requirements describe the key operations that the system must perform. These include user registration and authentication, project creation and management, task assignment, task status updates, Kanban board visualization, dashboards, and real-time chat communication.

Non-functional requirements define the quality attributes of the system, such as security, performance, responsiveness, and reliability. The system ensures secure authentication using encrypted passwords and role-based access control, while also maintaining a responsive user interface accessible across different devices.

The following table presents the detailed software requirements for the Opero Project Management System, outlining each requirement along with its description and importance.

Software Requirements

Req. No	Requirement	Essential / Desirable	Description of Requirement	Remarks
RS1	User Registration & Login	Essential	Users must be able to register and log in securely with encrypted passwords.	Basic requirement for access control.
RS2	Role-Based Access Control	Essential	System supports Admin, Project Manager, and Team Member roles.	Ensures controlled permissions.
RS3	Project Creation & Management	Essential	Users can create, edit, and delete projects with titles, descriptions, and deadlines.	Core functionality.

Req. No	Requirement	Essential / Desirable	Description of Requirement	Remarks
RS4	Task Creation & Assignment	Essential	Users can create tasks under projects and assign them to team members.	Required for workflow.
RS5	Task Status Management	Essential	Tasks can be marked as Pending, In Progress, or Completed.	Enables progress tracking.
RS6	Kanban Board	Essential	Users can drag and drop tasks across status columns.	Improves usability.
RS7	Dashboard View	Essential	Displays project progress, statistics, and deadlines.	Enhances productivity.
RS8	Calendar & Scheduling	Essential	Shows deadlines, milestones, and schedules on a calendar interface.	Improves planning.
RS9	Commenting System	Essential	Users can comment on tasks for discussion and collaboration.	Supports teamwork.
RS10	Notifications	Desirable	Users receive alerts for task updates, deadlines, comments, and mentions.	Improves engagement.
RS11	Analytics & Reports	Desirable	Generates charts and reports on progress and completion rate.	Useful for managers.
RS12	Project Templates	Desirable	Admin can create templates for commonly repeated project types.	Saves time.
RS13	Team Management	Essential	Managers can add or remove team members from specific projects.	Controls collaboration.
RS14	Responsive UI	Essential	System should function properly on mobile, tablet, and desktop.	Accessibility.

Req. No	Requirement	Essential / Desirable	Description of Requirement	Remarks
RS15	Security & Validation	Essential	Protects system from vulnerabilities and validates all inputs.	Critical safety requirement.
RS16	Backup & Recovery	Desirable	Supports scheduled backups and recovery to avoid data loss.	Reliability.
RS17	Admin Dashboard	Essential	Admin can manage users, permissions, reports, and system settings.	Platform oversight.
RS18	Error Handling	Essential	Displays user-friendly error messages for failed operations.	Improves user experience.

Table 2.1: Software Requirements for Opero Project Management System

2.5 Feasibility Analysis

2.5.1 Technical Feasibility

Technical feasibility assesses whether a project can be successfully developed and implemented using the available technologies, development tools, infrastructure, and technical expertise. It involves evaluating whether the selected technologies are suitable for building a scalable, secure, and efficient system.

The technologies chosen for the Opero project are highly feasible and align effectively with its objectives. The frontend is developed using React.js, which provides a responsive and dynamic Single Page Application (SPA) architecture. React enables efficient component-based development, real-time UI updates, and seamless navigation between dashboards, projects, tasks, and chat modules.

The backend is implemented using Node.js and Express.js, which provide a lightweight and scalable server-side environment for handling RESTful APIs, authentication, and business logic processing. MongoDB is used as a NoSQL database, offering flexible document-based storage for users, projects, tasks, and chat messages. Data modeling and validation are handled using Mongoose, ensuring structured schema design.

Authentication and security are implemented using JSON Web Tokens (JWT) and bcrypt password hashing, ensuring secure access control and protected routes. Real-time communi-

cation is enabled through Socket.io, allowing instant message exchange within project-specific chat rooms without requiring page refresh.

The development process is supported using Git and GitHub for version control and collaboration, ensuring structured development and efficient team coordination. The selected technology stack is widely adopted in the industry, well-documented, and supported by strong community resources. Therefore, the technical implementation of Opero is feasible, scalable, secure, and suitable for modern web application development.

2.5.2 Economical Feasibility

Economic feasibility evaluates whether a project is cost-effective and financially viable. It considers development expenses, deployment costs, maintenance requirements, and long-term sustainability.

The Opero project demonstrates strong economic feasibility, as it is built primarily using open-source and freely available technologies. React.js, Node.js, Express.js, and MongoDB Community Edition are free to use, significantly reducing development costs. Socket.io is also open-source, enabling real-time communication without additional licensing expenses.

Development tools such as Visual Studio Code, Git, and GitHub provide free plans that support efficient coding, version control, and collaboration. Deployment can be managed using affordable or free-tier cloud services such as Render, Vercel, or MongoDB Atlas free tier, minimizing hosting costs during the initial stages.

Maintenance costs are relatively low, as updates primarily involve feature enhancements, bug fixes, or UI improvements within the existing architecture. The modular design of the system ensures that new features can be added without major structural changes, reducing future redevelopment expenses.

Overall, Opero is a cost-effective and sustainable solution for project management and team collaboration, suitable for academic use as well as potential real-world deployment.

2.5.3 Operational Feasibility

Operational feasibility evaluates whether the project aligns with operational requirements and whether it can be smoothly integrated into real-world organizational workflows. It examines how effectively the system performs its core functions and whether it is practical, user-friendly, and efficient for its intended users.

The operational feasibility of the Opero project assesses how efficiently the system manages user authentication, project creation, task tracking, and real-time team communication. These features are designed to operate seamlessly through the integration of a structured MERN stack architecture. The frontend, developed using React.js, provides an intuitive and responsive interface with dynamic dashboards, Kanban-style task boards, and integrated chat functionality.

The component-based design ensures smooth navigation and real-time updates without requiring page refreshes.

The backend, built using Node.js and Express.js, efficiently handles API requests, authentication processes, project operations, and task management. Secure authentication is implemented using JSON Web Tokens (JWT), ensuring that only authorized users can access protected features. MongoDB serves as the database system, providing flexible and scalable data storage for users, projects, tasks, and chat messages.

Real-time communication is enabled using Socket.io, allowing team members to send and receive messages instantly within project-specific chat rooms. Task updates and workflow transitions are reflected dynamically, ensuring transparency and smooth collaboration among team members. Role-Based Access Control (RBAC) ensures that Admin, Manager, and Team Member roles operate within defined permissions, maintaining structured project management.

Overall, Opero demonstrates strong operational feasibility. Its streamlined architecture, secure authentication mechanisms, structured task workflow, and real-time collaboration features ensure practical usability in both academic and organizational environments. The system is user-friendly, scalable, and capable of supporting efficient project management operations.

2.6 System Environment

2.6.1 Software Environment

React.js (Frontend Framework):

The frontend development of Opero is carried out using React.js, a powerful JavaScript library for building dynamic and responsive user interfaces. React enables the implementation of a Single Page Application (SPA), ensuring smooth navigation without page reloads. It supports a component-based architecture, allowing reusable UI components such as dashboards, project panels, task boards, and chat interfaces. React Router is used for navigation between different pages such as login, registration, and project views. The framework ensures real-time UI updates when tasks are created, modified, or moved across workflow stages, enhancing user experience and responsiveness.

Node.js and Express.js (Backend Framework):

Node.js serves as the server-side runtime environment for Opero, enabling efficient handling of asynchronous operations. Express.js is used to design RESTful APIs that manage authentication, user operations, project management, task handling, and message storage. The backend follows a modular architecture, separating routes, controllers, models, and middleware for better maintainability and scalability. Middleware functions handle authentication verification, request validation, and error handling to ensure secure and structured backend operations.

MongoDB with Mongoose (Database Management):

MongoDB is used as the NoSQL database system for storing application data, including users, projects, tasks, and chat messages. It stores data in a flexible document-based format, allowing efficient handling of dynamic project information. Mongoose is used as an Object Data Modeling (ODM) library to define schemas, enforce validation rules, and manage relationships between collections. Automatic timestamping ensures tracking of record creation and updates, maintaining data consistency.

Socket.io (Real-Time Communication):

Socket.io is integrated into the system to enable real-time communication between team members. It establishes WebSocket connections between the client and server, allowing instant message broadcasting within project-specific chat rooms. This ensures seamless collaboration without requiring page refreshes. Messages are also stored in the database to maintain persistence and allow future retrieval.

Authentication and Security Tools:

JSON Web Tokens (JWT) are used for secure authentication and session management. Passwords are encrypted using bcrypt hashing before being stored in the database. Environment variables are managed using dotenv to securely store sensitive information such as database connection strings and secret keys.

Development and Version Control Tools:

The project is developed using Visual Studio Code as the primary code editor. Git is used for version control, and GitHub is used for repository management and collaboration. These tools ensure structured development, tracking of code changes, and effective teamwork.

2.6.2 Hardware Environment

Processor (CPU): Intel Core i5 (12th Generation)

Memory (RAM): 16 GB

Storage: 512 GB SSD

Operating System: Windows 11

Internet Requirement: Stable broadband connection for API communication and real-time messaging

2.7 Actors and Roles

The Opero system involves three primary actors: Admin, Manager, and Team Member. Each actor has specific responsibilities and access permissions within the system to ensure structured project management and secure collaboration.

Actor Name: Admin

Role:

- Manage user accounts including creation, updating, and deletion.
- Assign roles such as Manager or Team Member to users.
- Create, update, and delete projects.
- Monitor overall system activities and project progress.
- Access all projects, tasks, and communication modules.
- Ensure system integrity and enforce access control policies.

Actor Name: Manager

Role:

- Create and manage projects assigned to them.
- Add or remove team members from projects.
- Create, assign, update, and monitor tasks within projects.
- Track task progress using the Kanban workflow (Pending, In Progress, Completed).
- Participate in real-time project-based chat communication.
- Monitor team performance and ensure timely completion of tasks.

Actor Name: Team Member

Role:

- View assigned projects and tasks.
- Update task status based on progress.
- Participate in real-time project chat discussions.
- Collaborate with other team members within assigned projects.
- Provide updates and feedback on assigned tasks.

3 SYSTEM DESIGN

3.1 Use case model

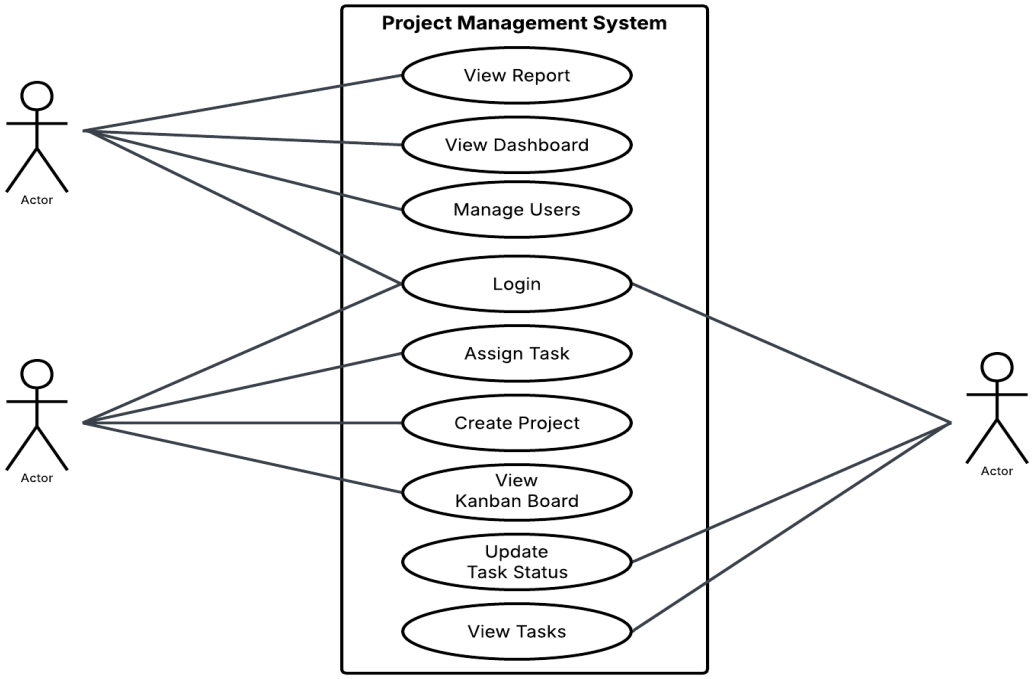


Figure 3.1: Use case Diagram

Use Case Diagram Explanation:

The use case diagram of the Opero Project Management System illustrates the interaction between three primary actors: Admin, Manager, and Member. The diagram represents the functional requirements of the system and shows how each actor interacts with different system operations.

The **Admin** actor has the highest level of access within the system. The Admin can log in to the system and perform administrative operations such as managing users, viewing the dashboard, and viewing reports. The Manage Users functionality allows the Admin to create, update, assign roles, or remove users from the system. The View Dashboard provides an overview of system activities and project summaries, while View Report allows monitoring of overall project progress and user activities.

The **Manager** actor is responsible for handling project-level operations. After logging into the system, the Manager can create new projects and assign tasks to team members. The Assign

Task use case allows the Manager to allocate responsibilities to specific members within a project. The Manager can also view the Kanban Board, which visually represents task statuses such as Pending, In Progress, and Completed. Additionally, the Manager can update task status and monitor overall task progress to ensure timely project completion.

The **Member** actor represents team members who work on assigned tasks. Members must log in to access the system. They can view tasks assigned to them and update task status as they progress through different workflow stages. Members primarily interact with the View Tasks and Update Task Status use cases. Their interaction ensures that project progress is continuously updated and visible to Managers and Admins.

The Login use case is common to all actors, as authentication is required before accessing any system functionality. The system boundary, labeled ”Project Management System,” defines the scope of all operations performed within Opero.

Overall, the use case diagram demonstrates role-based access control and structured interaction between users and system functionalities, ensuring secure and organized project management operations.

3.2 Activity Diagram

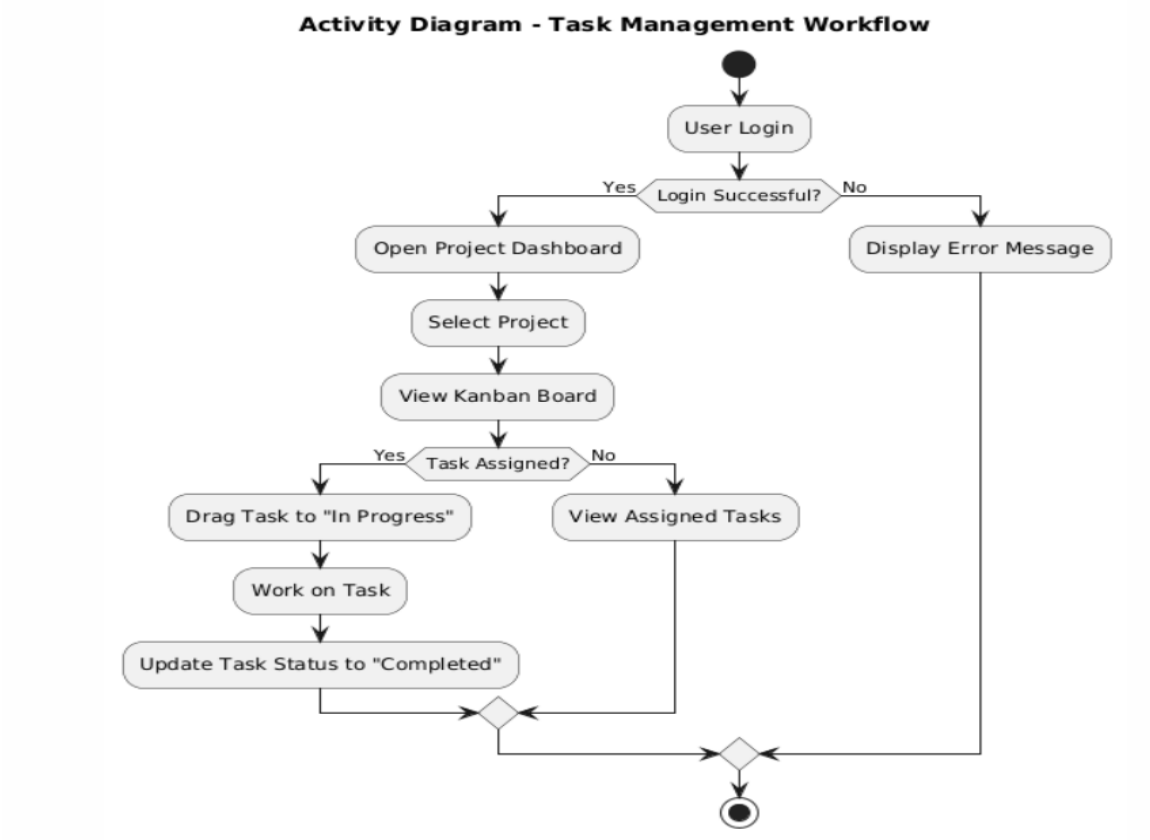


Figure 3.2: activity diagram

Activity Diagram Explanation:

The activity diagram represents the Task Management Workflow within the Opero Project Management System. It illustrates the sequence of actions performed by a user while interacting with projects and tasks in the system.

The workflow begins with the **User Login** activity. The system verifies the user credentials and checks whether the login is successful. If the login attempt fails, the system displays an appropriate error message and terminates the process. If the login is successful, the user is directed to the Project Dashboard.

From the dashboard, the user selects a specific project to work on. After selecting the project, the system displays the Kanban Board, which visually represents the tasks categorized under different stages such as Pending, In Progress, and Completed.

At this stage, the system checks whether a task is assigned to the user. If a task is assigned, the user drags the task from the Pending stage to the In Progress stage, indicating that work has started. The user then works on the task and, upon completion, updates the task status to Completed. This ensures that the workflow reflects real-time progress.

If no task is assigned to the user, the system allows the user to view assigned tasks or remain in the dashboard without making status changes.

Finally, once the task is updated or no further action is required, the workflow reaches the end state. The diagram clearly demonstrates the logical flow of task handling, including decision points, task transitions, and completion conditions.

Overall, the activity diagram highlights the structured workflow of task management in Opero, ensuring transparency, accountability, and systematic progress tracking within projects.

3.3 Sequence Diagram

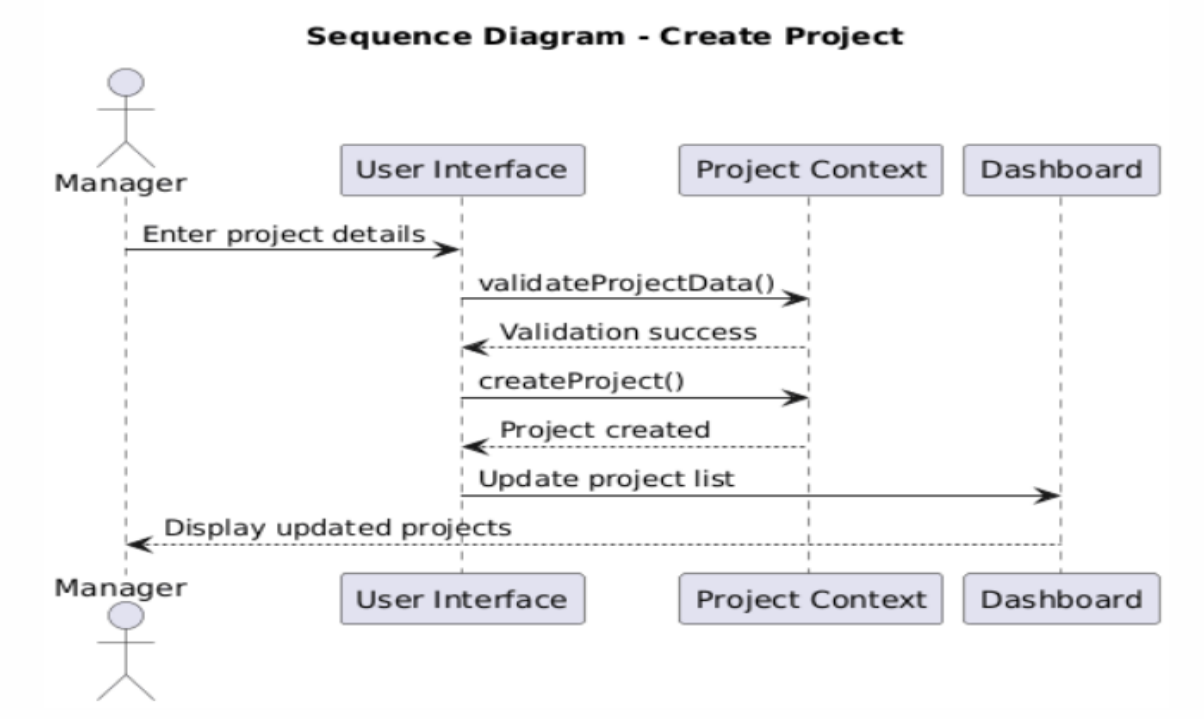


Figure 3.3: sequence diagram

Sequence Diagram Explanation:

The sequence diagram illustrates the process of creating a new project within the Opero Project Management System. It shows the interaction between the Manager (actor) and the internal system components, including the User Interface, Project Context, and Dashboard.

The process begins when the Manager enters project details through the User Interface. These details typically include the project name, description, and assigned members. Once the information is submitted, the User Interface sends the project data to the Project Context component for validation using the function validateProjectData().

The Project Context verifies whether the entered data meets the required validation criteria, such as checking for empty fields or incorrect formats. If the validation is successful, a confirmation message is returned to the User Interface indicating validation success.

After successful validation, the createProject() function is invoked within the Project Context. This function processes the creation request and stores the new project information in the system's database. Once the project is successfully created, a confirmation response is sent back to the User Interface.

Following project creation, the system updates the project list displayed on the Dashboard. The Dashboard retrieves the updated project data and refreshes the display to include the newly created project.

Finally, the Manager is shown the updated project list on the Dashboard, confirming that the project has been successfully created and integrated into the system.

Overall, the sequence diagram demonstrates the step-by-step interaction between the actor and system components, highlighting data validation, project creation logic, and real-time dashboard updates within the Opero system.

3.4 Class Diagram

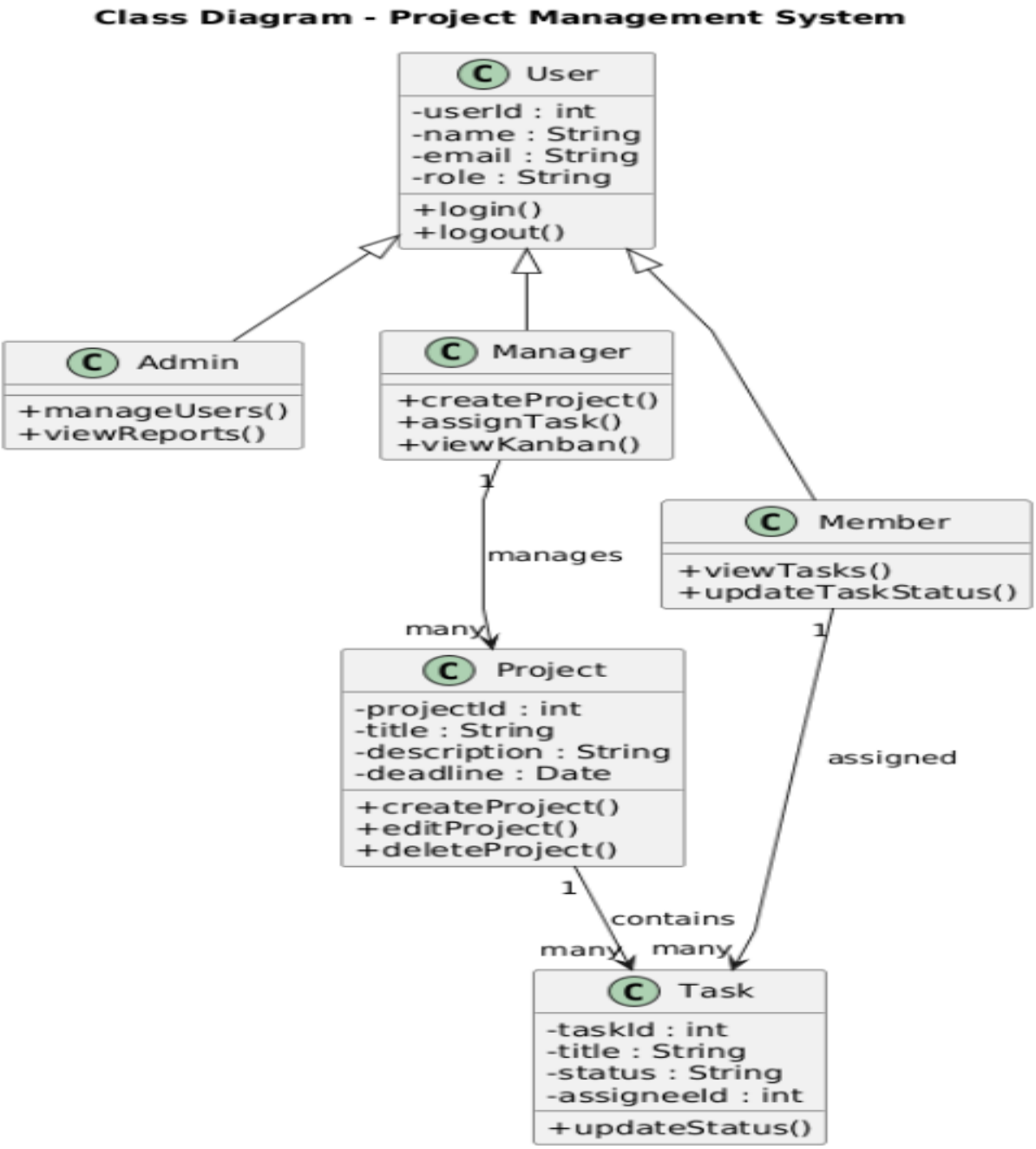


Figure 3.4: class diagram

Class Diagram Explanation:

The class diagram represents the structural design of the Opero Project Management System. It illustrates the main classes, their attributes, methods, and the relationships between them.

The central class in the system is the **User** class. It contains common attributes such as userId, name, email, and role. It also defines basic operations including login() and logout(). The User class acts as a parent class, demonstrating the concept of inheritance.

Three specialized classes inherit from the User class: **Admin**, **Manager**, and **Member**. These subclasses extend the base functionality of the User class by adding role-specific operations.

The **Admin** class includes methods such as manageUsers() and viewReports(), allowing administrative control over users and system-level monitoring.

The **Manager** class includes methods such as createProject(), assignTask(), and viewKanban(). Managers are responsible for creating projects and managing task workflows.

The **Member** class includes operations such as viewTasks() and updateTaskStatus(). Members interact primarily with assigned tasks and update their progress.

The **Project** class represents individual projects within the system. It includes attributes such as projectId, title, description, and deadline. The class defines operations like createProject(), editProject(), and deleteProject(). A Manager manages multiple projects, establishing a one-to-many relationship between Manager and Project.

The **Task** class represents tasks within a project. It includes attributes such as taskId, title, status, and assignedId. The updateStatus() method allows modification of task progress. Each Project contains multiple Tasks, establishing a one-to-many relationship between Project and Task.

Additionally, each Task is assigned to one Member, while a Member can be assigned multiple tasks, demonstrating another one-to-many relationship.

Overall, the class diagram illustrates object-oriented principles such as inheritance, association, and aggregation. It clearly defines system structure, role hierarchy, and relationships between users, projects, and tasks, ensuring organized and scalable system design.

3.5 Project Design

Project design is the process of defining the overall structure, workflow, and functional behavior of a system to ensure efficiency, usability, and scalability. It involves planning system architecture, user roles, data organization, interface layout, and interaction logic to provide a smooth and productive user experience. A well-designed project management system focuses on clarity, collaboration, security, and structured workflow progression. It ensures that users can manage tasks, monitor progress, and communicate effectively within a centralized platform.

In Opero, the project design focuses on integrating structured project management with real-time team collaboration in a web-based environment. The system is built using a modular MERN stack architecture that separates frontend, backend, and database layers to maintain scalability and maintainability. The user interface is designed using React.js to provide a clean dashboard, Kanban-style task board, and project-specific chat system, ensuring intuitive navigation and real-time updates.

The design emphasizes role-based interaction, where Admins manage users and oversee system activities, Managers create projects and assign tasks, and Team Members update task progress and participate in collaboration. The Kanban workflow (Pending, In Progress, Completed) ensures transparency and clear visualization of task progression. Real-time communication through Socket.io enhances coordination among team members within each project workspace.

Security and data management are core components of the design. Authentication is implemented using JSON Web Tokens (JWT), and passwords are securely encrypted using bcrypt. MongoDB schemas are structured to maintain organized storage of users, projects, tasks, and messages.

Overall, the design of Opero balances usability, functionality, and collaboration, creating an efficient, secure, and scalable project management system suitable for academic demonstration and real-world organizational use.

3.5.1 UI Design

The User Interface (UI) of the Opero Project Management System is designed to be clean, intuitive, and responsive. The interface enables users to efficiently interact with different system modules such as authentication, project management, task tracking, and real-time communication. Each interface is designed to support role-based access control, ensuring that Admins, Managers, and Members can perform their respective tasks efficiently.

1. Login Page

The login page allows registered users to securely access the system by entering their email and password. Authentication is verified using JWT-based authentication, ensuring secure system access.

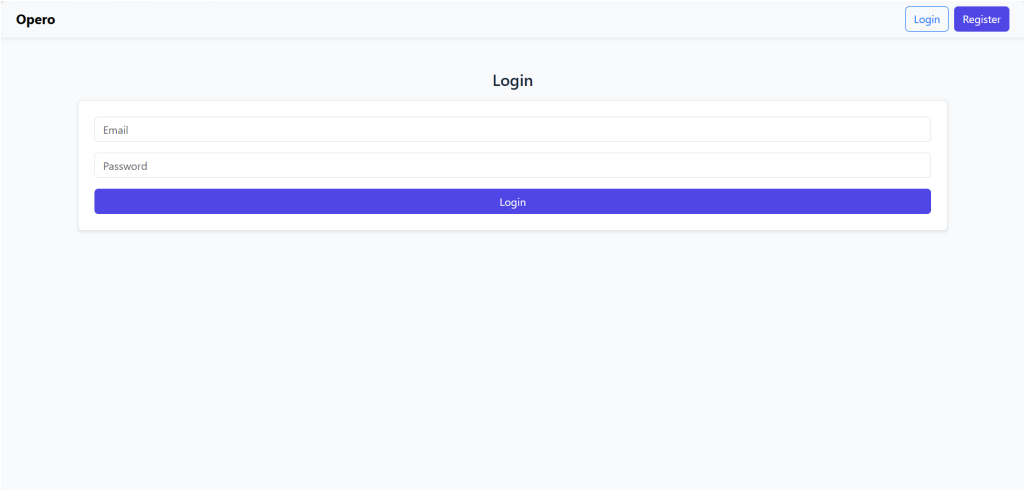


Figure 3.5: Login Page

2. Registration Page

The registration page allows new users to create an account by providing basic information such as name, email, password, and role. After successful registration, users can log in to access the system.

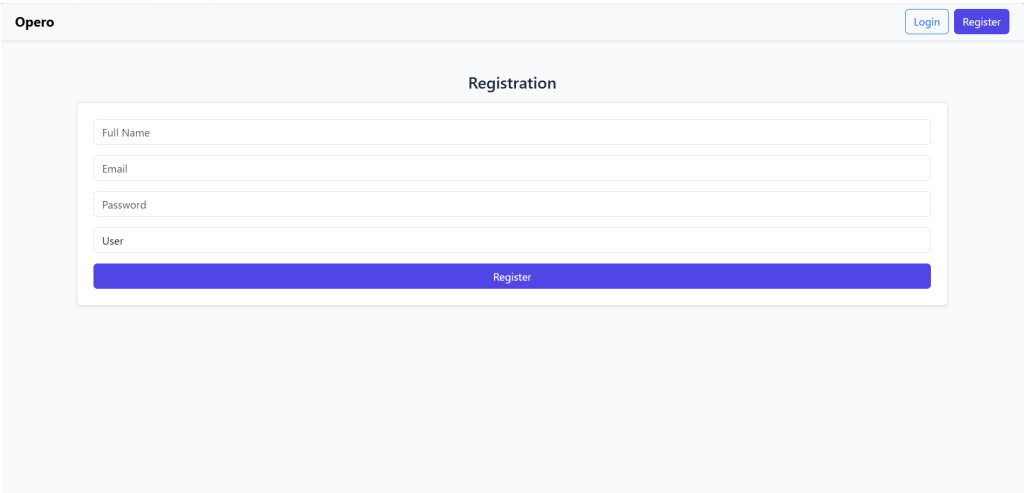


Figure 3.6: Registration Page

3. Admin Dashboard

The Admin dashboard provides an overview of the entire system. Admin users can manage user accounts, view system activities, and monitor project progress through this interface.

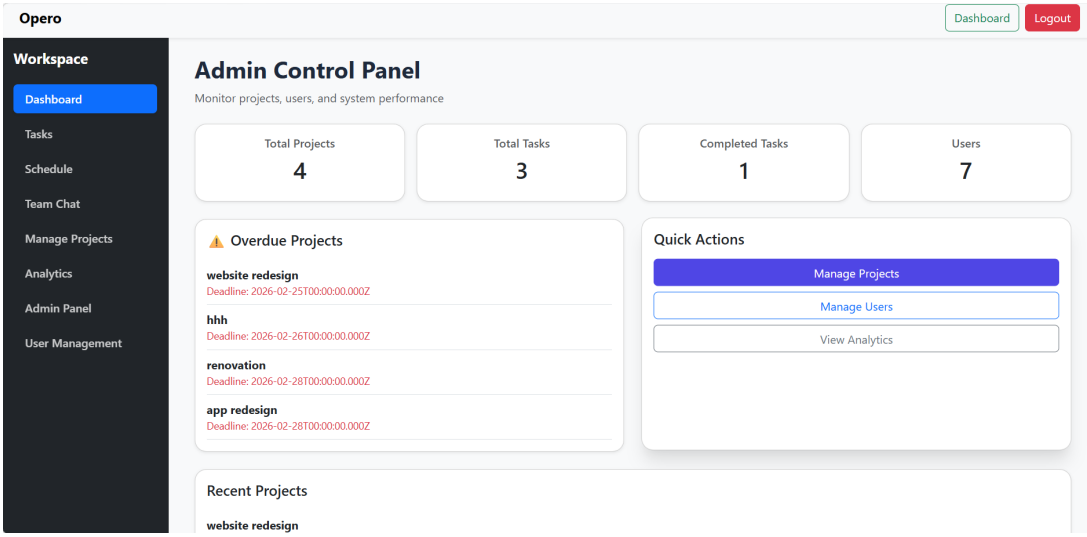


Figure 3.7: Admin Dashboard

4. Manager Dashboard

The Manager dashboard allows project managers to create projects, assign tasks to team members, and track project progress using task boards and reports.

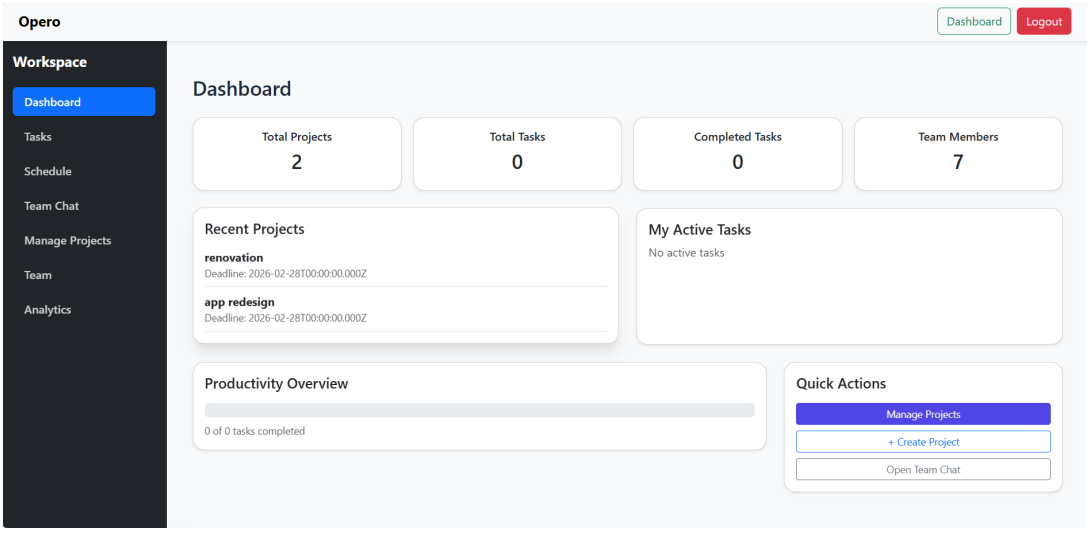


Figure 3.8: Manager Dashboard

5. Project Creation Page

This interface allows managers to create new projects by entering project details such as project name, description, and assigned team members.

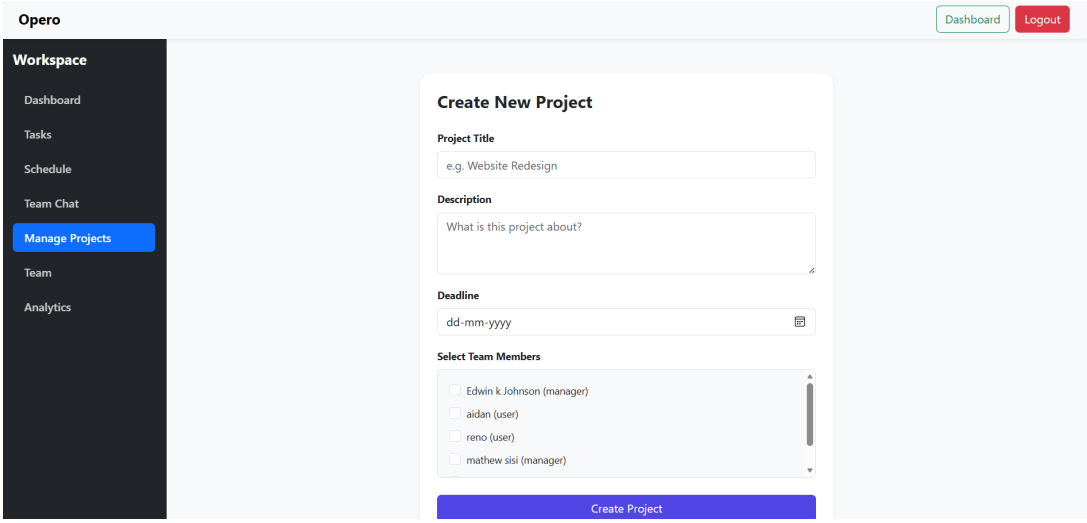


Figure 3.9: Project Creation Page

6. Kanban Board

The Kanban board provides a visual representation of task workflow. Tasks are organized into different columns such as Pending, In Progress, and Completed, allowing users to easily track project progress.

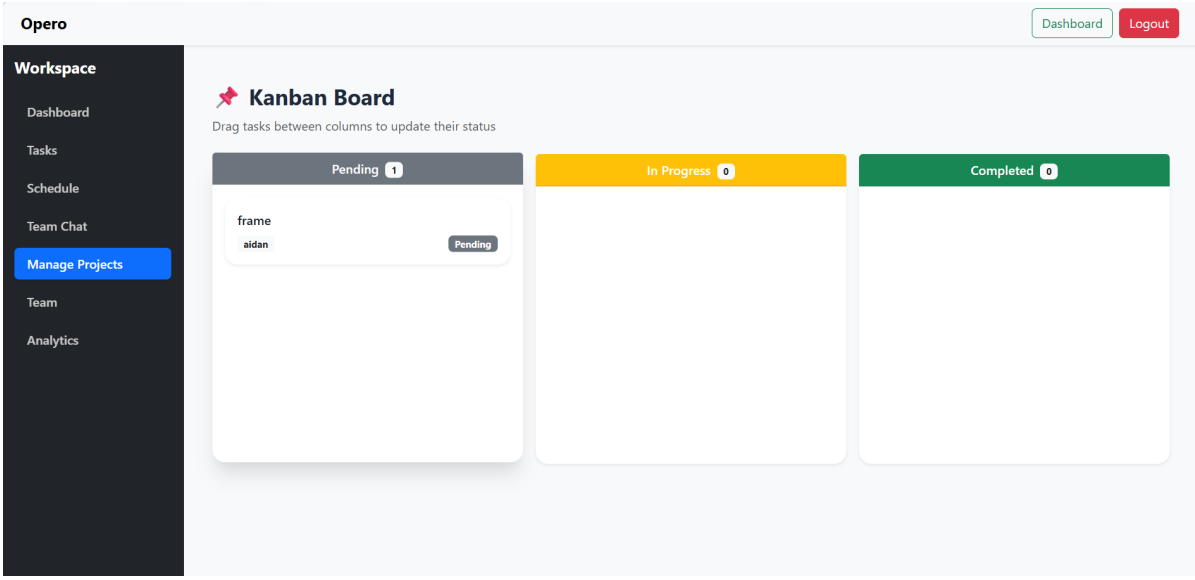


Figure 3.10: Kanban Task Board

7. Task Management Page

The task page allows managers to assign tasks to team members and update task details. Team members can view their assigned tasks and update their status based on progress.

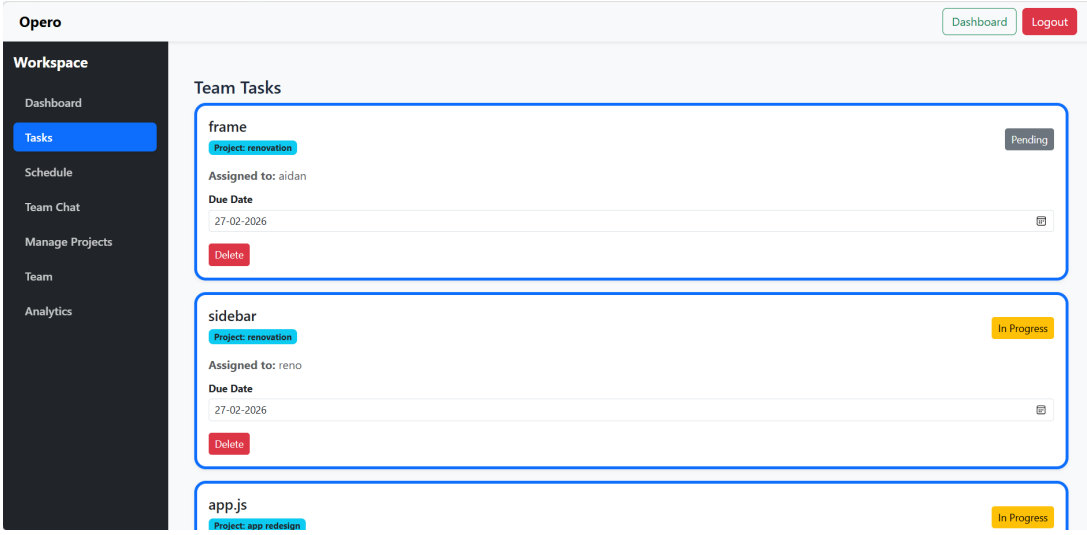


Figure 3.11: Task Management Page

8. Chat Interface

The chat interface enables real-time communication among project members. Messages are exchanged instantly using Socket.io, allowing effective collaboration and discussion within each project workspace.

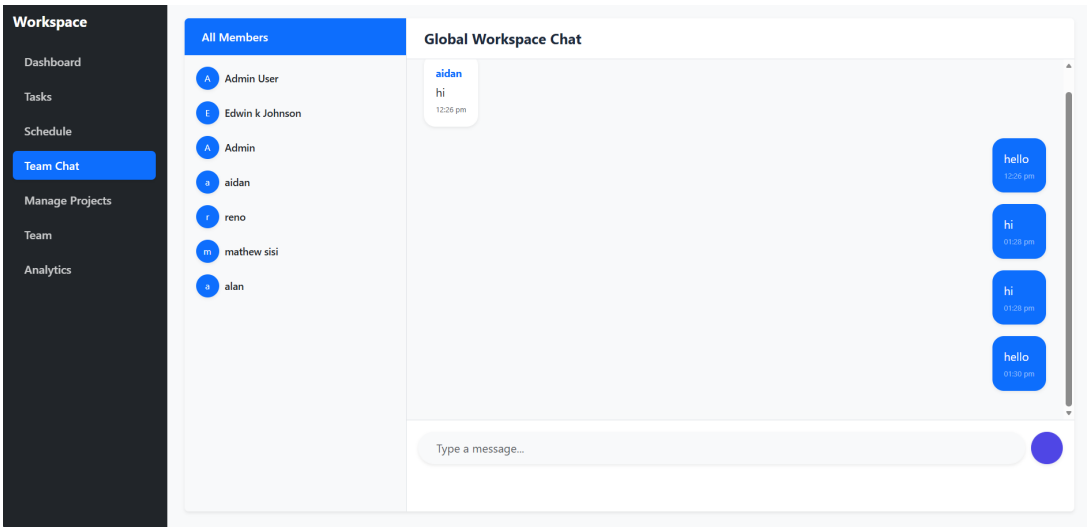
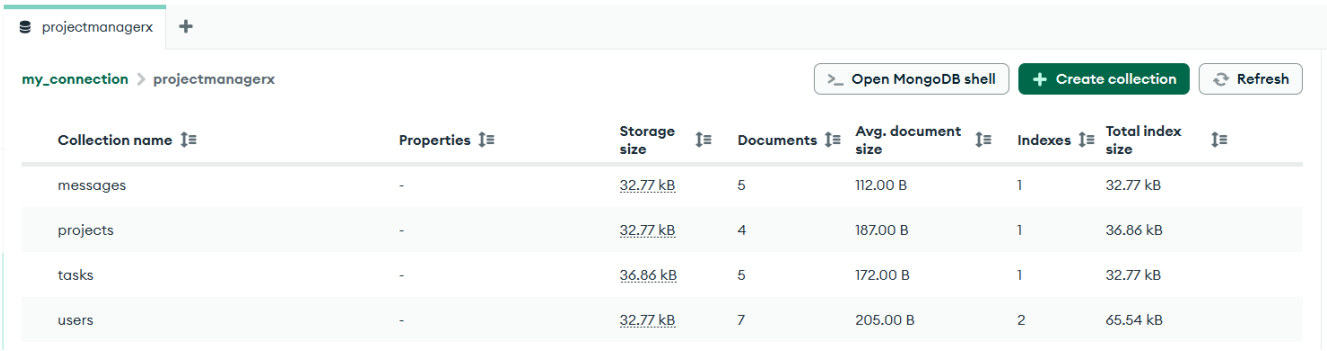


Figure 3.12: Project Chat Interface

3.5.2 Database Design

The database design of Opero is implemented using MongoDB with Mongoose schemas. The system stores data related to users, projects, tasks, and chat messages in structured collections.

The User collection stores authentication and role information, the Project collection stores project details, the Task collection tracks assigned tasks and their statuses, and the Message collection stores chat messages exchanged between team members. This schema-based design ensures organized data storage and efficient data retrieval.



Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
messages	-	32.77 kB	5	112.00 B	1	32.77 kB
projects	-	32.77 kB	4	187.00 B	1	36.86 kB
tasks	-	36.86 kB	5	172.00 B	1	32.77 kB
users	-	32.77 kB	7	205.00 B	2	65.54 kB

Figure 3.13: databases in MongoDB

4 TESTING

Testing is a crucial phase in software development that ensures the system functions correctly, is free from major defects, and provides a reliable experience for users. Proper testing helps identify errors in system functionality, security vulnerabilities, and performance issues before deployment. In the Opero Project Management System, different testing techniques are applied to ensure that the application operates smoothly and meets its functional requirements. The primary testing approaches used include unit testing, integration testing, system testing, and performance testing.

Unit Testing: Unit testing focuses on verifying the functionality of individual components of the system. In Opero, unit testing is applied to modules such as user authentication, project creation, task assignment, and chat messaging. Each module is tested independently to ensure that it performs the intended function correctly. For example, the authentication module is tested to verify that users can successfully register and log in, while invalid credentials are properly rejected.

Integration Testing: Integration testing ensures that different modules of the system work together seamlessly. In Opero, integration testing verifies the interaction between the frontend and backend components. For example, when a manager creates a project through the user interface, the request is sent to the backend API, processed by the server, and stored in the MongoDB database. Integration testing confirms that these components communicate correctly without errors.

System Testing: System testing evaluates the entire application as a complete system to ensure that all modules function together as expected. In Opero, this includes testing the complete workflow from user login to project management, task updates, and real-time chat communication. The system is tested under different user roles such as Admin, Manager, and Team Member to verify that role-based access control and permissions function correctly.

Performance Testing: Performance testing ensures that the application runs efficiently under different conditions and workloads. In Opero, performance testing evaluates how the system handles multiple users interacting simultaneously, including project updates, task movements on the Kanban board, and real-time chat messaging. The goal is to ensure that the system maintains fast response times and stable performance without crashes or delays.

4.1 Unit Testing

Unit testing focuses on testing individual components of the Opero system, such as authentication, project management, task operations, and real-time messaging, to ensure that each module functions correctly in isolation. By testing each component independently, developers can detect errors early in development and maintain system reliability. Unit testing ensures that important features like user login, project creation, task updates, and chat messaging behave as

expected.

Test Case ID	Component	Test Description	Expected Result	Status
UT01	User Registration	Verify that a new user can successfully register with valid details	User account is created and stored in the database	Pass
UT02	User Login	Verify that users can log in using valid credentials	User is authenticated and redirected to dashboard	Pass
UT03	Role-Based Access	Ensure that Admin, Manager, and Member roles access only authorized features	Users can only access features permitted by their role	Pass
UT04	Project Creation	Verify that a manager can create a new project with valid project details	Project is successfully stored and displayed in dashboard	Pass
UT05	Task Assignment	Ensure that managers can assign tasks to team members	Task is assigned and visible to the selected member	Pass
UT06	Task Status Update	Verify that team members can update task status on the Kanban board	Task moves correctly between Pending, In Progress, and Completed	Pass
UT07	Real-Time Chat	Verify that messages sent in project chat are delivered instantly	Message appears immediately for all project members	Pass
UT08	Project Dashboard	Ensure that the dashboard displays updated project and task information	Dashboard shows correct project summaries and task status	Pass

Table 4.1: Unit Test Cases for Opero Project Management System

4.2 Integration Testing

Integration testing ensures that different modules of the Opero Project Management System interact correctly with each other without errors. It verifies that components such as the user interface, backend APIs, database operations, and real-time communication services function together as a unified system.

For example, integration tests verify whether user authentication correctly communicates with the database, whether project creation from the frontend is properly stored in MongoDB through the backend API, and whether real-time chat messages are transmitted correctly using Socket.io. By conducting integration testing, developers can identify and resolve issues that arise when multiple system components interact, improving overall system stability and user experience.

Test Case ID	Component	Test Description	Expected Result	Status
IT01	Login UI and Authentication API	Verify if login credentials from the UI are correctly validated by the backend API	User is authenticated and redirected to the dashboard	Pass
IT02	Registration and Database	Ensure that new user registration details are correctly stored in the MongoDB database	New user account is successfully created and stored	Pass
IT03	Project Creation and Database	Verify that projects created through the frontend are stored correctly in the database	Project appears in the dashboard after creation	Pass
IT04	Task Management and Kanban Board	Ensure that updating task status reflects correctly on the Kanban board	Task moves correctly between Pending, In Progress, and Completed columns	Pass
IT05	Real-Time Chat and Socket.io	Verify that chat messages are transmitted instantly between project members	Messages appear instantly for all connected users	Pass
IT06	Dashboard and Project Data	Ensure that dashboard displays updated project and task information	Dashboard reflects correct project summaries and task updates	Pass

Table 4.2: Integration Test Cases for Opero Project Management System

4.3 System Testing

System testing evaluates the entire Opero Project Management System as a complete application to ensure that all components work together correctly. It verifies the system's overall functionality, performance, security, and user experience. Unlike unit testing or integration testing, system testing focuses on validating the behavior of the entire application including the user interface, backend services, database operations, and real-time communication features.

System testing ensures that all modules such as authentication, project management, task tracking, dashboard updates, and real-time chat operate together as expected under normal usage conditions.

Test Case ID	Component	Test Description	Expected Result	Status
ST01	System Launch	Verify that the Opero application loads correctly without errors	Application starts and displays login page successfully	Pass
ST02	User Interface	Check whether all UI components such as buttons, forms, and navigation menus function correctly	UI elements respond properly to user actions	Pass
ST03	User Authentication	Ensure users can register, log in, and access the system based on their roles	Users are authenticated and redirected to their respective dashboards	Pass
ST04	Project and Task Workflow	Verify that projects can be created, tasks assigned, and task status updated on the Kanban board	Project and task workflow functions correctly	Pass
ST05	Real-Time Communication	Check whether chat messages are delivered instantly between project members	Messages are transmitted and displayed in real-time	Pass
ST06	Performance Testing	Ensure the system handles multiple user actions without crashes or delays	Application runs smoothly with stable performance	Pass

Table 4.3: System Test Cases for Opero Project Management System

4.4 Performance Testing

Performance testing ensures that the Opero Project Management System runs efficiently under different usage conditions. It evaluates system responsiveness, page loading speed, API response time, server resource usage, and real-time communication performance. A well-performing web application should respond quickly to user actions, handle multiple users simultaneously, and maintain stable performance without crashes or delays.

Performance testing is conducted to verify that features such as authentication, project creation, task updates, and real-time chat operate smoothly even when multiple users interact with the system.

Test Case ID	Component	Test Description	Expected Result	Status
PT01	Page Load Time	Measure time taken to load login page and dashboard	Pages load within 2-3 seconds	Pass
PT02	API Response Time	Verify response time of backend APIs for authentication and project operations	API responses received within 2 seconds	Pass
PT03	Database Performance	Check the speed of storing and retrieving project and task data from MongoDB	Database queries execute efficiently without delays	Pass
PT04	Real-Time Chat Latency	Verify message delivery time between users using Socket.io	Messages appear instantly for all connected users	Pass
PT05	Server Resource Usage	Monitor CPU and memory usage during multiple user interactions	Server resources remain within acceptable limits	Pass

Table 4.4: Performance Test Cases for Opero Project Management System

5 DEPLOYMENT

5.1 Folder Structure

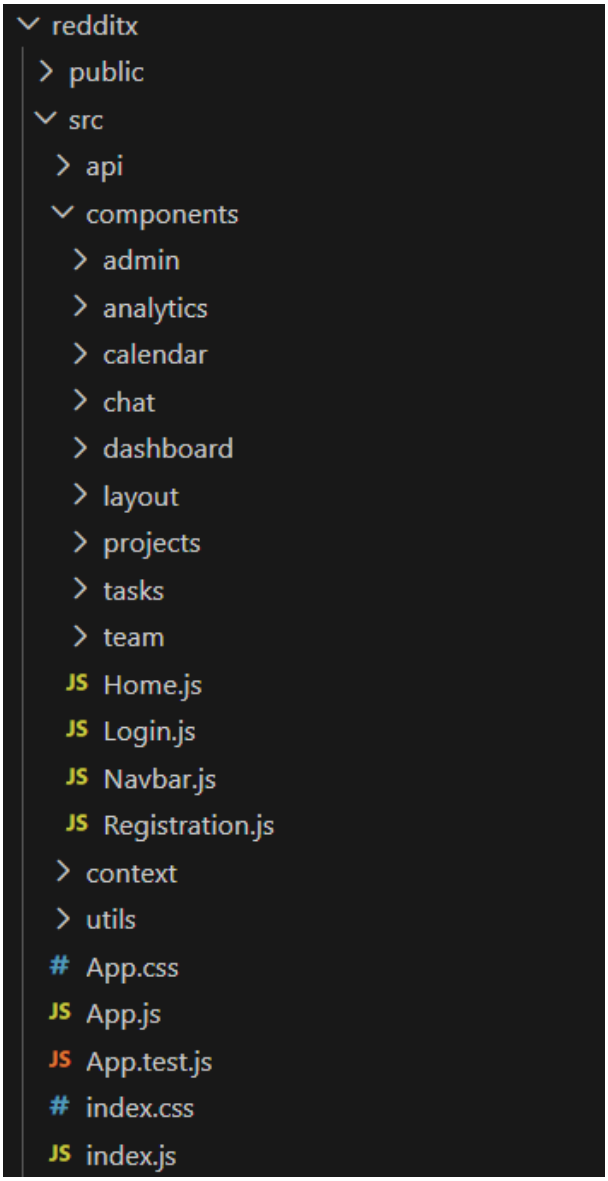


Figure 5.1: src files

6 GIT HISTORY

To ensure proper version management and easy distribution, all game builds have been uploaded to Git. This allows for efficient tracking of updates, bug fixes, and improvements over time. By using Git, different versions of the game can be accessed, making it easier to roll back changes if needed. The repository serves as a centralized location for storing and sharing the game builds, ensuring that the latest version is always available for testing and distribution.

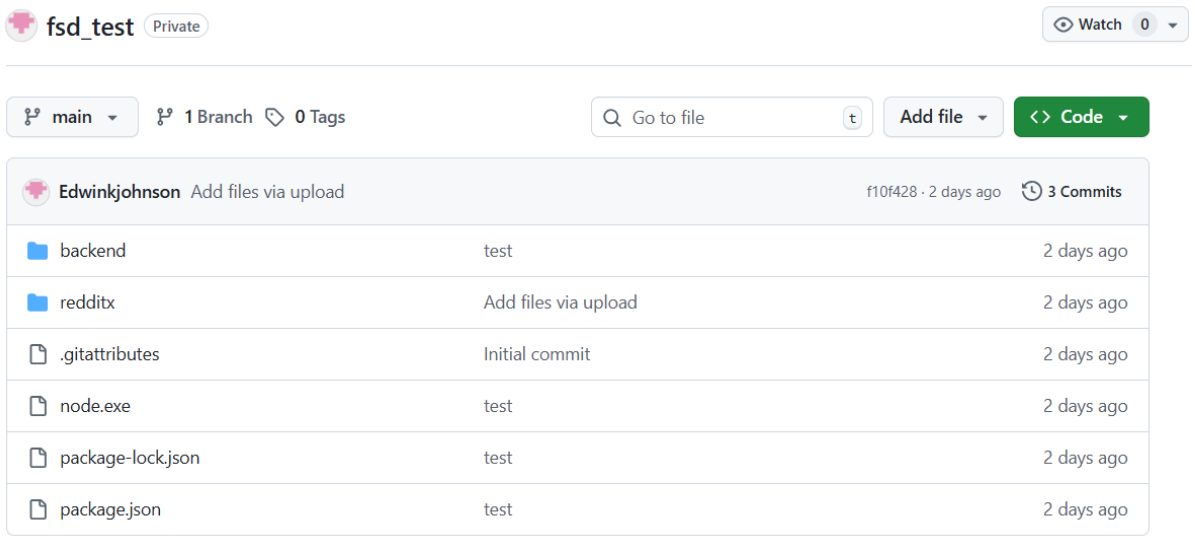


Figure 6.1: Builds Uploaded

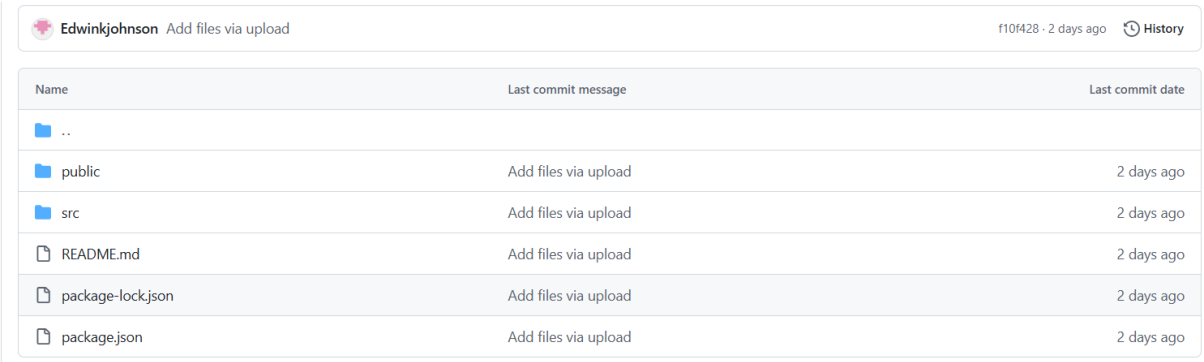


Figure 6.2: Files and folders inside the builds

7 CONCLUSIONS

The Opero Project Management System was successfully designed and developed as a web-based platform that supports efficient team collaboration, task management, and project tracking. The system demonstrates how modern web technologies can be integrated to create a scalable and user-friendly project management environment.

The application provides essential features such as user authentication, role-based access control, project creation, task assignment, Kanban-based workflow visualization, and real-time communication among team members. By separating responsibilities between Admin, Manager, and Team Member roles, the system ensures structured workflow management and controlled access to system functionalities. The Kanban board implementation helps visualize task progress clearly, improving productivity and transparency within project teams.

The system was implemented using the MERN stack architecture, where React.js was used to build an interactive and responsive user interface, Node.js and Express.js handled the backend APIs and business logic, and MongoDB provided a flexible and scalable database for storing users, projects, tasks, and messages. Real-time communication was achieved using Socket.io, allowing team members to collaborate effectively within project workspaces.

Extensive testing was conducted including unit testing, integration testing, system testing, and performance testing to ensure that the application functions correctly and reliably. The results of the testing process confirm that the system performs efficiently, maintains data consistency, and provides a smooth user experience.

Overall, the Opero Project Management System successfully achieves its objective of providing a structured platform for project planning, task tracking, and team collaboration. The system demonstrates the practical application of modern web development frameworks and serves as a strong academic implementation of a real-world project management solution.